



QUIETPLACE

ADVANCED PROGRAMMING APPLICATIONS

COURSE CODE: CCS2304

12TH PROJECT

Prepared by :

Malak Mostafa: 241017093

Tasneem Mohamed: 241017093

Mai Mohamed: 251005125

Mona Khaled: 241014463

Maram Mohamed: 241007361

Jowairya Ayman: 241004121

Nourhan Yasser: 241004766

Presented To:

Dr. Maida Mahmoud

Eng. Mohammed Fouad

PROJECT IDEA

QuietPlace is a web application designed to help users find quiet places for studying, working, or relaxing based on their noise level preferences. The platform allows users to search for nearby locations using an interactive map, filter places by noise levels and amenities, submit their own noise readings to keep data current, and save their favorite spots for quick access. A personalized recommendation system analyzes each user's noise preferences and suggests matching places, making it easier to discover suitable environments. The application includes user authentication with secure registration and login, an admin dashboard for managing users and places, and a session-based system that tracks user activity. Built with Spring Boot and MySQL, the project demonstrates full-stack development concepts including RESTful API design, database relationships, geospatial calculations using the Haversine formula, and the Model-View-Controller architecture pattern.

FEATURES IMPLEMENTED

System Feature Breakdown

A. User Registration & Authentication

Overview

The system provides a complete user authentication module that enables secure user account management and session-based access control.

Functional Description

- Users can register using **username, email, and password**.
- Passwords are securely stored using **BCrypt hashing**, ensuring that plain-text passwords are never saved in the database.
- Authentication is implemented using a **manual session-based mechanism** rather than Spring Security's default authentication workflow.
- Upon successful login:
 - A server-side session is created.
 - User identity is stored in the session context.
- Logout functionality invalidates the active session and removes authentication data.

Technical Concepts

- Session Management
- Password Encryption (BCrypt)
- Authentication Flow
- DTO-based request validation

B. Place Management

Overview

This module represents the central data model of the application and manages all information related to locations (places).

Functional Description

Each place entity stores:

- Name and address

- Geographic coordinates (latitude & longitude)
- Noise level indicators
- Supported purposes (study, work, relaxation, etc.)
- Available amenities
- Aggregated statistics:
 - Average noise level
 - Average crowdedness

Capabilities

- Full **CRUD operations** available for administrators.
- Public users can:
 - Retrieve places
 - Filter results
 - Perform nearby location searches.

Technical Concepts

- Spring Data JPA ORM mapping
- Repository Pattern
- RESTful API Design
- Domain Modeling

C. Favourites Feature

Overview

The favourites system allows users to personalize their experience by saving preferred locations.

Functional Description

- Authenticated users can add places to a favourites list.
- The system prevents duplicate favourite entries.
- Users may:
 - Add favourites
 - Remove favourites
 - Check favourite status
 - Retrieve total favourite counts

Technical Concepts

- User–Place relational mapping
- Many-to-Many relationship handling
- Session-based authorization checks
- Business rule validation

D. User Profile & Preferences Management

Overview

This module provides profile visualization and personalization features for users.

Functional Description

The user profile page displays:

- User information
- Account creation date
- Last login timestamp

Users can update:

- Preferred noise level

- Default search radius
- Default geographic location
- Automatic location detection toggle

Additional planned features:

- Change password
- Account deletion

Technical Concepts

- Profile Management
- Preference Persistence
- User Personalization
- Entity Update Operations

E. Exploration & Search Module (Home Page)

Overview

The exploration module enables users to discover suitable locations based on customizable search parameters.

Functional Description

Users can search using:

- Location
- Search radius
- Noise level preference
- Intended purpose
- Required amenities

Results are presented through:

- Interactive **Leaflet map visualization**
- Structured list view of matching places

Technical Concepts

- Geospatial Search Logic
- Dynamic Filtering
- Map Integration (Leaflet.js)
- MVC Controller–View interaction

F. Administrative Management Module

Overview

The administration module provides system management capabilities restricted to users with administrative privileges.

Functional Description

Admin dashboard includes:

- System statistics:
 - Total users
 - Total places
 - Recently registered users
- User management:
 - User listing
 - Account deletion
- Place management:
 - Create

- Update
- Delete operations

Access control is enforced using:

`@PreAuthorize("hasRole('ADMIN')")`

ensuring role-based authorization.

Technical Concepts

- Role-Based Access Control (RBAC)
- Method-Level Security
- Authorization Rules
- Administrative Dashboard Design

G. Noise Reading Submission & Verification Module (Early Stage)

Overview

This module introduces community-driven environmental data collection.

Functional Description

- Users can submit noise measurements for a specific place.
- Submitted data includes:
 - Noise level
 - Crowdedness rating
- Each submission is stored for later verification.
- Administrators can validate readings before acceptance.
- System enforces a **daily submission limit of five readings per user**.
- Verified readings contribute to calculating:
 - Average noise level
 - Average crowdedness per place.

Technical Concepts

- User-generated data workflow
- Data validation and moderation
- Aggregation algorithms
- Rate limiting logic

SYSTEM DESIGN (UML DIAGRAM)

SYSTEM ARCHITECTURE

The QuietPlace system is designed using a **layered architecture** based on the **Spring Boot MVC (Model–View–Controller) pattern**.

This architecture separates responsibilities across multiple layers, improving maintainability, scalability, and code organization.

The system consists of the following layers:

1. Presentation Layer

This layer represents the user interface of the system.

Responsibilities:

- Displays web pages using **Thymeleaf templates**
- Collects user input
- Sends HTTP requests to controllers

Examples:

- Login Page
- Home Page
- Profile Page
- Admin Dashboard

2. Controller Layer

Controllers handle incoming HTTP requests and act as an entry point to the backend.

Responsibilities:

- Receive requests from the client
- Validate input data
- Call appropriate service methods
- Return views or API responses

Examples:

- AuthController
- PlaceController
- UserController
- AdminApiController
- NoiseReadingController

3. Service Layer

The service layer contains the core business logic of the application.

Responsibilities:

- Authentication and session handling
- Recommendation logic
- Favourite management
- Noise aggregation calculations
- User preference processing

Examples:

- UserService
- PlaceService
- RecommendationService

4. Repository Layer

The repository layer communicates directly with the database using **Spring Data JPA**.

Responsibilities:

- CRUD operations
- Database queries
- Entity persistence

Examples:

- UserRepository
- PlaceRepository
- NoiseReadingRepository
- RecommendationRepository

5. Database Layer

Stores all persistent data required by the system.

Main Entities:

- Users

- # UML CLASS DIAGRAM



The following sequence describes how a login request is processed:

- ## YEAR IN REVIEW

3. Input validation is performed using DTO objects.
4. UserService retrieves the user from the database via UserRepository.
5. The password is verified using BCrypt password encoder.
6. If authentication succeeds, a user session is created.
7. The user is redirected to the home page.

Example: Place Search Flow

1. The user submits search filters from the home page.
2. PlaceController receives the request.
3. Filters are forwarded to PlaceService.
4. The service queries PlaceRepository.
5. Matching places are returned.
6. Results are displayed on the map and list view.

EXPLANATION OF USER ROLES AND PERMISSIONS

Overview:

The QuietPlace system implements a role-based access control (RBAC) model with two distinct user roles: Regular User and Administrator (Admin). Each role has specific permissions and access levels within the application.

User Roles:

Regular User:

A Regular User is a standard registered user who can search for places, submit noise readings, and manage their personal preferences.

Access Level: Authenticated User Permissions:

Action	Permission
----- -----	
View home page	✓ Yes
Search for places	✓ Yes
Filter places by noise level	✓ Yes
Filter places by purpose	✓ Yes
Filter places by amenities	✓ Yes
View place details	✓ Yes
Submit noise readings	✓ Yes
Save places to favorites	✓ Yes
Remove places from favorites	✓ Yes
View personal profile	✓ Yes
Update personal preferences	✓ Yes
Change password	✓ Yes
Delete own account	✓ Yes

| Logout | ☒ Yes |

Restrictions:

- Cannot add new places to the system
- Cannot edit or delete existing places
- Cannot view or manage other users
- Cannot verify noise readings
- Cannot access admin pages

Administrator (Admin):

An Administrator has full access to the system, including user management, place management, and content moderation.

Access Level: Privileged Authenticated User Permissions

(includes all Regular User permissions plus):

| Action | Permission |

|-----|-----|

| View admin dashboard | ☒ Yes |

| View all registered users | ☒ Yes |

| Edit user information | ☒ Yes |

| Delete user accounts | ☒ Yes |

| Change user roles | ☒ Yes |

| Add new places | ☒ Yes |

| Edit existing places | ☒ Yes |

| Delete places | ☒ Yes |

| Verify/unverify places | ☒ Yes |

| View all noise readings | ☒ Yes |

| Verify noise readings | ☒ Yes |

| Delete/reject noise readings | ☒ Yes |

| View system analytics | ☒ Yes |

| Moderate user reviews | ☒ Yes |

Permission Matrix:

| Feature | Regular User | Admin |

|-----|-----|-----|

| **Account Management** | | |

| Register account | ☒ | ☒ |

| Login | ☒ | ☒ |

| View own profile | ☒ | ☒ |

| Edit own profile | ☒ | ☒ |

| Delete own account | ☒ | ☒ |

| View all users | ☒ | ☒ |

| Edit any user | ☒ | ☒ |

Delete any user	✗	✓	
Place Management			
View places	✓	✓	
Search/filter places	✓	✓	
Add new place	✗	✓	
Edit place	✗	✓	
Delete place	✗	✓	
Verify place	✗	✓	
Noise Readings			
Submit noise reading	✓	✓	
View own readings	✓	✓	
View all readings	✗	✓	
Verify readings	✗	✓	
Delete readings	✗	✓	
Favorites			
Save to favorites	✓	✓	
Remove from favorites	✓	✓	
View favorites	✓	✓	
Admin Functions			
Access admin dashboard	✗	✓	
View system analytics	✗	✓	
Moderate content	✗	✓	

Access Control Implementation:

Authentication vs Authorization:

Term	Definition	In QuietPlace
-----	-----	-----
Authentication	Verifying user identity	Login with email and password
Authorization	Determining what user can do	Role-based permissions

How Access is enforced:

Using Spring Security Annotations:

```

```java
// Only Admin can access
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/users")
public List<User> getAllUsers() { ... }

```

```

//Authenticated users (both USER and ADMIN) can access
@PreAuthorize("isAuthenticated()")

```

```
@PostMapping("/favorites/add")
public ResponseEntity<?> addToFavorites() { ... }
```

```
// Public access (no login required)
@GetMapping("/explore")
public String explorePage() { ... }
```

